

Learn JavaScript: Beginner's Course

Lesson 2: Variables and Data Types

Variables and Data Types - Study Notes

Key Concepts

- **Declaration:** Using `let` (reassignable) and `const` (block scoped, not reassignable) to create variables.
- **Primitive Types:** Numbers, strings, booleans – immutable values stored directly.
- **Reference Types:** Arrays and objects – store collections or key value pairs; variables hold a reference to the data.
- **Type Flexibility:** JavaScript is dynamically typed; a variable can change its type at runtime (except `const` bindings).
- **Scope & Hoisting:** `let`/`const` are block scoped and not hoisted in the same way as `var`.

Summary

In JavaScript, variables are named containers for data. You declare them with `let` when you expect the value to change, or with `const` when the binding should stay constant (though the contents of an object or array can still be mutated). The language provides several built-in data types: **numbers** for numeric values (integers and floating point), **strings** for textual data wrapped in single, double, or back tick quotes, and **booleans** for true/false logic. For more complex structures, **arrays** store ordered lists accessed by numeric indices, while **objects** store unordered collections of key value pairs (properties). Understanding how to declare, initialize, and manipulate these types is foundational for writing any JavaScript program.

Important Points

- Use `const` by default; switch to `let` only if reassignment is needed.
- Numbers follow IEEE 754 floating point rules; beware of precision issues (e.g., `0.1 + 0.2 !== 0.3`).
- Strings are immutable; methods like `toUpperCase()` return a new string.
- Booleans result from comparison operators (`===`, `>`, etc.) and logical operators (`&&`, `||`).
- Array literal syntax: `[item1, item2, ...]`; access with `arr[index]`.
- Object literal syntax: `{ key: value, ... }`; access via dot (`obj.key`) or bracket (`obj["key"]`) notation.
- `null` represents an intentional absence of value; `undefined` means a variable has been declared but not assigned.
- `typeof` operator reveals the type of a value (`"number"`, `"string"`, `"boolean"`, `"object"` for arrays/objects, `"function"`).

Examples

Example 1 – Declaring and Using Primitives

```
```\njavascript\nlet count = 5;           // number, can be reassigned\nconst pi = 3.14159;      // number, constant binding
```

```

let name = "Ada"; // string
const isReady = true; // boolean
count = count + 1; // OK: let allows reassignment
// pi = 3; // TypeError: Assignment to constant variable
name = name.toUpperCase(); // strings are immutable; returns new string
console.log(count, pi, name, isReady);
// Output: 6 3.14159 ADA true
...

```

## Example 2 – Working with Arrays and Objects

```

````javascript
const scores = [88, 92, 79]; // array of numbers
scores.push(85);             // mutating the array is allowed
console.log(scores[0]);      // 88 (first element)
const person = {
  firstName: "Ada",
  lastName: "Lovelace",
  age: 36,
  isProgrammer: true
};
person.age = 37;              // updating a property
person["city"] = "London";    // adding a new property via bracket notation
delete person.isProgrammer;   // removing a property
console.log(person);
// Output: { firstName: 'Ada', lastName: 'Lovelace', age: 37, city: 'London' }
...

```

Quick Review

1. **When should you prefer `const` over `let`?**
2. **What is the result of `typeof []` and why?**
3. **Explain the difference between `null` and `undefined`.**
4. **How do you add a new element to the end of an array without mutating the original array?**
5. **Give one way to check if a variable holds a boolean value.**

Further Reading

- **Truthy & Falsy values** – understanding how non boolean types are coerced in conditionals.
- **Template literals** – multi line strings and expression interpolation (``Hello ${name}``).
- **Destructuring assignment** – extracting values from arrays and objects concisely.
- **Map, Set, and WeakMap/WeakSet** – advanced collection types beyond plain arrays/objects.
- **Immutability patterns** – using `Object.freeze`, spread syntax, or libraries like `Immutable.js` for safer state handling.

